

HealthConnect Clinic -- Final ML Pipeline & Engineering Package

Week 8 -- Machine Learning Engineering Track

Programme	AnalystLab Africa Experience Lab
Week	Week 8 -- Final Integration, Presentation & Project Showcase
Project	HealthConnect Clinic Experience Lab
Track	Machine Learning Engineering
Prepared by	Marie Claire Niyomugenga
Document version	1.0

1. Week 7 → Week 8 Transition

- Main Week 7 output: 12 executed tests covering reproducibility, preprocessing, and invalid/malformed input handling; a refactored model-integration interface (NoShowModelInterfaceV2); and a real cross-track test of Alia's (Data Science) booking_month feature.
- Most important testing result: the Week 6 baseline is exactly reproducible on retraining, and booking_month, once actually tested, did not improve performance (ROC-AUC 0.682 vs. 0.683) -- an honest negative result, not adopted.
- Major issue discovered: two real bugs -- no dtype validation on numeric input (a malformed value crashed with a raw internal error), and an interface hardcoded to one feature set, which could not host a model needing a different one (proven by wrapping a booking_month model and watching it fail with "columns are missing: {'booking_month'}").
- What was refined or improved: NoShowModelInterfaceV2 -- per-model feature contracts (feature_fn/feature_cols/numeric_cols), dtype validation with clear error messages, and logging on every scoring call.
- Which component is now considered ready: the model-integration interface itself is final and demonstration-ready -- proven to work correctly with two internally-built models and, via a tested reproduction, with Alia's independently-derived methodology.
- What still needs to be integrated: a real serialized model file (tuned_gradient_boosting_model.pkl) was received from Alia, but inspection found it is a raw GradientBoostingClassifier -- not a scikit-learn Pipeline -- with no feature_names_in_ attached, so the exact column order and preprocessing needed to produce its expected 33 input features cannot be determined from the artifact alone. Also still open: the Data Analytics track's segment-level findings, if shared; a fairness pass on gender/age.
- Which track(s) are involved in final integration: Data Science (Alia Al-Qadri) directly; Project Management for final readiness coordination.
- What I intend to demonstrate in the final presentation: the full pipeline flow from raw appointment data to a risk score; the interface correctly hosting two structurally different models through one identical call; and the honest cross-track testing story -- Alia's spec tested, one feature rejected on evidence, one real integration gap found and closed.

2. Part 1 -- Final Integration Readiness

Requirement	Your Response
What is my final component?	NoShowModelInterfaceV2, plus the integrated ingestion → feature engineering → training → inference pipeline it sits inside.
What problem does it address?	Lets any HealthConnect no-show model -- regardless of which track built it, or which exact features it needs -- be scored consistently and reliably inside one production-shaped workflow, without rework each time the underlying model changes.
What is its current status?	Tested and validated: 12 real test cases executed in Week 7 (11 passed outright, 1 genuine issue found and then fixed and retested). Reproducible on retraining. Demonstration-ready. A real artifact from Data Science was received and inspected in Week 8; it cannot yet be safely wrapped due to a missing column/preprocessing mapping (see Section 7).
What did I improve during Week 7?	Added numeric dtype validation (a malformed value now raises a clear, named error instead of a raw internal one) and generalised the interface to accept a per-model feature contract (feature_fn/feature_cols/numeric_cols) instead of one hardcoded global feature set.
Which other track(s) does it depend on?	Data Science (Alia Al-Qadri) -- for the final model or model specification this interface will ultimately host.
Which track(s) depend on my output?	Data Science (needs this interface to make their model usable inside the actual pipeline); Project Management (needs final ML Engineering status for the project readiness assessment).
What output will I provide?	A documented, tested model-integration interface (NoShowModelInterfaceV2), the integrated pipeline it lives in, and this Final ML Pipeline & Engineering Package as evidence of readiness.
What output do I need from another track?	A serialized artifact was received (tuned_gradient_boosting_model.pkl). What is still needed: either the fitted preprocessing object (ColumnTransformer/encoder+scaler) used to produce its 33 input features, or the exact ordered list of those 33 column names -- neither is recoverable from the classifier object alone.
What evidence demonstrates readiness?	The Week 7 Testing & Validation Record (12 logged tests); the booking_month cross-track test and its retest; the interface's backward-compatibility retest against the original Week 6 models.
What limitations remain?	No real fitted Data Science artifact is integrated yet -- only a tested reproduction of Alia's methodology. No fairness evaluation has been run on gender/age. The Sunday-appointment / Knowledge Base inconsistency from Week 5 remains unresolved.

3. Final Integrated Pipeline

The pipeline's shape is unchanged in structure since Week 5/6 -- ingestion → engineer_features() → patient-grouped train/test split → training → inference -- but the model-integration point at the end of that chain is now a generalised, tested contract rather than a single hardcoded classifier.

- Ingestion: loads HealthConnect_Appointment_Data.csv, unchanged since Week 5.
- Feature engineering: engineer_features() -- missing-value handling, prior_no_show_rate with a division-by-zero guard, and the lead-time bucket -- validated directly in Week 7 testing, not just assumed correct.
- Split: GroupShuffleSplit on patient_id, ~80/20, random_state fixed -- confirmed to give zero patient overlap between train and test.
- Training: LogisticRegression selected on evidence in Week 6 over RandomForest and GradientBoosting; confirmed exactly reproducible in Week 7.
- Model integration: NoShowModelInterfaceV2 -- the final component of this package -- wraps any fitted model behind one identical predict_risk()/get_metrics() call, validates its inputs and outputs, and logs its activity.

4. Repository & Configuration

- Repository structure: ingestion, feature engineering, training and the model-integration interface (src/models/interface.py) are organised as separate modules, consistent with the structure established in Week 5/6.
- Dependencies: pandas and scikit-learn only -- unchanged since Week 5. No new dependency was needed for either the Week 7 dtype validation or the Week 8 final packaging.
- Configuration: config.yaml / requirements.txt unchanged and confirmed accurate as of Week 7's review.
- README.md: updated with a Week 7 changelog section documenting the interface refactor and the booking_month finding (see repository).

5. Testing Summary

12 tests were executed and logged in Week 7 via an automated test harness, covering training reproducibility, preprocessing components, interface input/output validation, and cross-track compatibility. 11 passed outright; 1 (a deliberately surfaced malformed-input scenario) failed as expected, was fixed, and passed on retest. Full detail, including exact expected/actual results, is in the Week 7 ML Pipeline Testing & Reliability Package.

- Reproducibility: PASS -- retrained baseline metrics identical to the original run (accuracy 0.629, ROC-AUC 0.683).
- Preprocessing edge cases (first-time patients, missing reminder_channel, unseen categorical values): all PASS.
- Invalid/unexpected input (single row, extra columns, malformed numeric value): PASS on the first two; the malformed-value case failed under the Week 6 interface and passed after the Week 7 fix.
- Cross-track compatibility with a model requiring booking_month: failed under the Week 6 interface (confirming a real gap), passed under NoShowModelInterfaceV2.
- Data Science artifact inspection (Week 8): tuned_gradient_boosting_model.pkl loads correctly and passes a mechanical predict_proba sanity check, but is confirmed NOT safely integrable without its missing preprocessing/column-order metadata -- a genuine finding, not a failed test.

6. Error Handling & Reproducibility

NoShowModelInterfaceV2 validates required raw columns, engineered feature columns, and numeric dtypes before scoring, raising a clear, named error at each failure point rather than allowing a malformed input to fail deep inside scikit-learn. Every scoring call is logged. The training step was independently retested in Week 7 and confirmed to produce identical metrics under the same fixed random seed and split -- the pipeline's results are not dependent on incidental session state.

7. HC-POD Mandatory Final Integration -- Data Science (Alia Al-Qadri)

- Track collaborated with: Data Science (Alia Al-Qadri).
- Dependency: the pipeline's model-integration layer needs a final model artifact from Data Science to host.
- Output received: Alia's full, confirmed methodology (Weeks 6-7, including booking_month), and, in Week 8, a real serialized model file -- tuned_gradient_boosting_model.pkl.
- Output provided: NoShowModelInterfaceV2, tested and validated end-to-end against her methodology, ready to host any correctly-specified model.
- Final integration activity: the artifact was loaded and inspected directly rather than assumed to be ready to wrap. Findings: it is a raw GradientBoostingClassifier (learning_rate=0.05, n_estimators=50, subsample=0.9), not a scikit-learn Pipeline, so it does not bundle her preprocessing (one-hot encoding, scaling). It expects exactly 33 input features (n_features_in_ = 33) but has no feature_names_in_ attribute -- it was fit on unnamed data, so neither the identity nor the order of those 33 columns can be recovered from the artifact itself. A mechanical sanity check (predict_proba on a zero-vector of the correct shape) confirmed the file itself loads correctly and is not corrupted -- the gap is specifically the missing column/preprocessing mapping, not a broken file.
- Issue/finding identified: a bare classifier, shared without its fitted preprocessing object or an ordered feature list, cannot be safely integrated -- wrapping it without that information would mean guessing at column order, which this project's entire Week 7 testing philosophy was built to avoid.
- Refinement/action taken: rather than guess, a specific, concrete follow-up request was sent back to Data Science -- either the fitted preprocessing object (ColumnTransformer/encoder+scaler), or the exact ordered list of the 33 trained-on column names. Alia confirmed this classifier object is the complete extent of what she currently has serialized.
- Retest result: integration is correctly blocked pending that information -- not silently worked around. This is documented here as the final, honest Week 8 status rather than a fabricated handoff.
- What changed as a result: the pipeline's integration point moved from "designed to be compatible" (Week 6), to "proven compatible against confirmed methodology" (Week 7), to "a real artifact received, mechanically validated, and a specific, well-defined integration gap identified and requested" (Week 8) -- a precise, actionable status rather than a vague "still pending."
- How the change improved the solution: this is exactly the kind of gap that would otherwise surface silently in production -- a model that loads fine but scores nonsense because its inputs are misaligned. Catching it here, before any wrapping was attempted, protects the pipeline's core testing principle: nothing is assumed correct until it is actually tested.
- Evidence: tuned_gradient_boosting_model.pkl itself; the model-inspection output (type, n_features_in_=33, feature_names_in_ absent, predict_proba sanity check); Slack thread in #hc-pod-03 (8 and 12 September, and the Week 8 follow-up); this Week 8 package.
- Contribution to the final presentation: this is the strongest cross-track evidence in the whole project -- a genuine artifact was received and genuinely tested, and a real, specific integration risk was

caught before it could cause silent harm, rather than the interface being assumed ready the moment a file arrived.

8. Known Limitations

- Alia's real fitted model (`tuned_gradient_boosting_model.pkl`) has been received but is not yet integrated: it is a bare classifier with no bundled preprocessing and no recoverable feature order, so it cannot be safely wrapped without additional information (the fitted preprocessing object or the exact 33-column list) that has been requested but not yet received.
- No fairness evaluation has been run on the model's use of gender/age features.
- The Sunday-appointment / Knowledge Base inconsistency identified in Week 5 remains unresolved.
- The Data Analytics track's segment-level findings have not been incorporated into the feature set, as they were not shared by submission time.

9. Final Integration Evidence & Presentation Materials

This package, the Week 7 Testing & Validation Record, the Slack thread with Alia (#hc-pod-03), and the repository (including `README.md` and `src/models/interface.py`) together constitute the evidence base for this track's contribution to the final HC-POD walkthrough and the individual video presentation.